Return-Aware Support for Apriori and FP-Growth on Online Retail Transactions

Group: [GROUP NAME]

[Member 1 name], contribution [xx]% [Member 2 name], contribution [xx]%

COSC-3026-F01 Data Mining and Data Warehousing, Final Report

Abstract. **Apriori and FP-Growth count every item that was in a basket at checkout, so an item the customer later returned still counts as bought. We link each cancellation line of Online Retail II to the purchase it reverses, define the kept basket k(t) of each purchase basket t, and replace support with retained support, the share of baskets in which every item of an itemset was bought and kept. Retained support is anti-monotone, so both algorithms prune on it unchanged. On 36,597 purchase baskets with a customer number, 88.6% of 17,974 cancellation lines link fully to earlier purchases, and 0.77% of basket items count as returned. Both algorithms give identical itemsets under both measures at all nine minimum supports, and our standard mode matches mlxtend. The return-aware version of Apriori takes 1.6 to 2.3 times as long as the baseline, while FP-Growth costs the same. At minimum support 0.005 the return-aware run reports 52 fewer itemsets (1.4%) and 50 fewer rules (1.9%), the top-50 rule lists are identical, and the Kendall rank correlation between confidence and retained confidence is 0.983. Rules removed by the return-aware run held on a later hold-out period as often as the rules both runs kept (81% against 80%), so re-ranking did not improve rule stability on this dataset. The itemset return rate is a weak but persistent signal: itemsets flagged on the first year had a median hold-out return rate of 0.012 against 0.004 for the others (one-sided Mann-Whitney $p < 0.001$).**

I. Introduction and Motivation

A. Purpose and aim

Online retailers mine purchase records for products that customers buy together and use the result for bundles, product placement and cross-selling suggestions. The usual tool is frequent itemset mining with Apriori [1] or FP-Growth [2]. Both count a basket as it stood at checkout. Returns happen later, and in the public Online Retail II dataset [3] they are stored as separate cancellation invoices whose number starts with C. Basket analyses of this retailer's data often delete these rows before mining. One public example keeps 396,470 of 541,909 rows of the earlier one-year release and then runs Apriori [4]. After this step the purchase of a returned item still counts. A pair of products that customers often buy and then send back therefore looks as strong as a pair they keep.

Returns are costly. Karl [5], citing the National Retail Federation and Appriss Retail, reports that US consumers returned $212 billion of merchandise bought online in 2022, 16.5% of online sales. Duong et al. [6] treat returns as an unavoidable cost of doing business and single out online marketplaces.

Our aim is to measure how much returns change the itemsets and association rules that Apriori and FP-Growth report, using a support measure that counts only items the customer kept.

B. Objectives

- Build a reproducible step that links each cancellation line to the purchase lines it reverses.
- Implement Apriori and FP-Growth with a switch between standard support (baseline) and retained support (return-aware), and check them against each other and against mlxtend [7].
- Compare the two measures on runtime, memory, the itemsets and rules reported, the return rate of reported patterns, and how well the top rules hold on a later hold-out period.

C. Uniqueness of the work

The change sits inside the support count, not in a separate post-processing filter. Because retained support is anti-monotone, both search procedures prune on it without extra bounds, which utility measures need [8]. Any difference in output therefore comes from returns alone. Prior work on returns forecasts or manages them [5], [6], [9], [10] or mines the return records themselves [11]. We did not find work that changes the support count of Apriori or FP-Growth to account for linked returns.

D. Summary of the work

We cleaned Online Retail II (1,067,371 rows), removed a block of 22,523 rows that appears in both sheets of the workbook, linked 17,974 cancellation lines to earlier purchases, and built full and kept baskets for 36,597 purchase invoices. We implemented both algorithms with both measures, passed 29 tests including exact agreement with mlxtend on real data, and ran seven experiments (E1 to E7) over nine minimum supports from 0.04 to 0.003. Two extra analyses (E4b, E5b) test whether return rates persist over time and whether the rules the two runs disagree on hold on the hold-out period.

II. Literature / Background Study

Apriori and FP-Growth. Agrawal and Srikant [1] introduced Apriori, which generates candidates level by level and prunes any candidate with an infrequent subset. Han et al. [2] introduced FP-Growth, which compresses the database into a prefix tree and mines it without candidate generation. Both define support on transactions as recorded. Neither has a notion of an item leaving a transaction after the fact.

Surveys and comparisons. Luna et al. [12] review 25 years of frequent itemset mining and note that cost grows exponentially with the number of items and that memory use is large. Bustio-Martínez et al. [13] review FPGA and GPU acceleration and state that mining in reasonable time on large

data is still open. Hunyadi et al. [14] compare Apriori and FP-Growth on e-commerce data in RapidMiner and report FP-Growth faster on large data. These works study speed. They take the transaction database as given and do not ask whether its contents are final.

Weighted and utility-based measures. Utility mining weights items by profit or quantity. Gan et al. [8] survey the area and state that utility is neither monotone nor anti-monotone, so methods need upper bounds to prune. TopHUI [15] mines the top-k high-utility itemsets in databases with both positive and negative utility, using a positive-and-negative utility list and threshold-raising strategies. He et al. [16] mine skyline frequent-utility itemsets that trade frequency against utility on massive data. A negative utility could encode a refund, but these methods attach it to the item value, not to a specific basket, and they need bound machinery that a support count does not.

Uncertain transactions. In uncertain databases an item belongs to a transaction only with some probability. Tong et al. [17] compare the expected-support and probabilistic definitions of a frequent itemset over such data, and Li et al. [18] add item weights to probabilistic frequent itemset mining. Our kept basket is a deterministic special case: after linking, an item is either kept or returned. These methods assume item probabilities are given and do not derive them from return records.

Deletions in streams. SpaceSaving± [19] finds frequent single items in a stream where a bounded fraction of insertions is later deleted, with a deterministic space bound. CICLAD [20] maintains closed itemsets over a sliding window and handles the removal of whole obsolete transactions. Both handle deletions, but of single items or whole transactions. A return removes one item from one earlier basket, which neither model expresses.

Product returns. Karl [5] and Duong et al. [6] review return forecasting and return research. Ambilkar et al. [9] review returns management as a supply chain process. Shang et al. [10] use transaction data to forecast return volumes. Li et al. [21] predict return intention from basket composition with a hypergraph of past purchases and returns, and SizeFlags [22] reduces size-related returns in fashion with a Bayesian model tested in production. These works predict or prevent returns. They do not change which co-purchase patterns a miner reports. Meißner et al. [11] apply frequent itemset mining with time series methods to the return records of a furniture shop. They mine returns themselves, not purchase baskets.

This dataset. Chen et al. [23] used the earlier one-year release of the same retailer's data for RFM customer segmentation. The analysis in [4] removes cancellations before running Apriori.

Summary of prior work. Frequent itemset research has optimized the search over a fixed database, and returns research has predicted or prevented returns. Measures that change support either break anti-monotonicity (utility) or assume given probabilities (uncertain data). Deletion-aware miners remove whole transactions or single items. The gap our work fills is a support count that uses linked returns, keeps anti-monotonicity, and leaves both search procedures unchanged.

III. Proposed Model

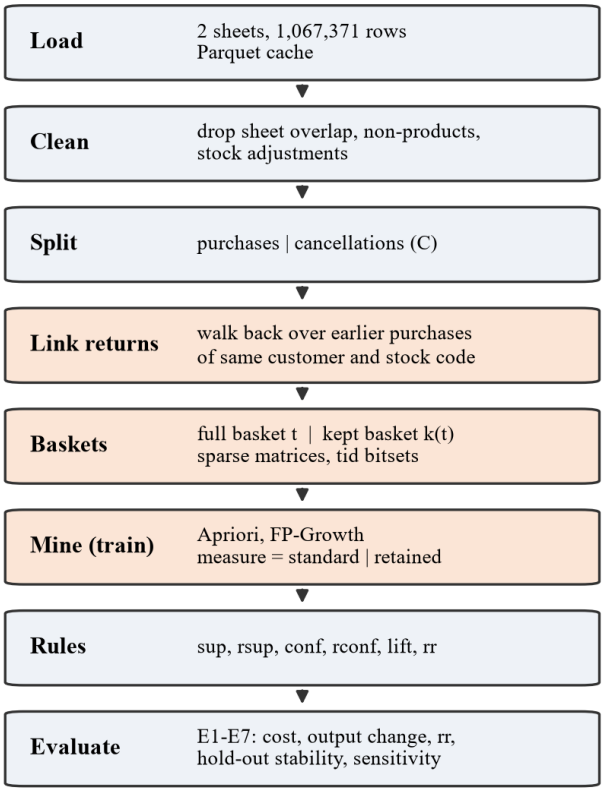


Fig. 1. Pipeline. Shaded steps are where the return-aware version differs from the baseline.

A. Definitions

Let D be the set of purchase baskets and, for a basket t, let k(t) ⊆ t be the items the customer did not return. Standard support is $\sup(X) = |\{t \in D : X \subseteq t\}| / |D|$ and retained support is $\mathrm{rsup}(X) = |\{t \in D : X \subseteq k(t)\}| / |D|$. The itemset return rate is $\mathrm{rr}(X) = 1 - \mathrm{rsup}(X) / \sup(X)$. For a rule $A \Rightarrow B$, standard confidence is $\mathrm{conf} = \sup(A \cup B) / \sup(A)$ and retained confidence is $\mathrm{rconf} = \mathrm{rsup}(A \cup B) / \mathrm{rsup}(A)$. Lift is conf / sup(B) for the baseline and rconf / rsup(B) for the return-aware run. If $X \subseteq Y$ and $Y \subseteq k(t)$, then $X \subseteq k(t)$, so $\mathrm{rsup}(Y) \leq \mathrm{rsup}(X)$. Retained support is anti-monotone, and $\mathrm{rsup}(X) \leq \sup(X)$ because $k(t) \subseteq t$. Every return-aware frequent itemset is therefore also a baseline frequent itemset at the same threshold.

B. Return linking

A cancellation line gives a customer, a stock code, a date and a quantity, but no reference to the original invoice. For each cancellation line we allocate its quantity to purchase lines of the same customer and stock code dated strictly before it, starting with the most recent and walking backwards until the quantity is used up. Cancellations are processed in date order, so one purchase is never returned twice. A line with no

customer number or no earlier matching purchase is an unlinked cancellation. Under the default partial-return policy "full", an item counts as returned in a basket only when the allocated quantity covers all of it. Under the policy "any", any allocation marks it returned.

C. Algorithms

Apriori generates candidates of size k by joining frequent itemsets of size k−1 that share a prefix, and prunes candidates with an infrequent subset [1]. We count candidates by intersecting vertical tid bitsets (one bit per basket, 64 baskets per machine word) and counting set bits. In return-aware mode each candidate gets two counts in the same pass, one on the bitsets of full baskets and one on kept baskets, and the miner prunes on the kept count.

FP-Growth scans the data twice to build a prefix tree ordered by item frequency and mines conditional pattern bases recursively, with the single-path shortcut [2]. In return-aware mode we build the tree from kept baskets, so node counts are retained counts and the mining step is unchanged. The standard count of each reported itemset comes afterwards from tid bitsets of the full baskets, so rr(X) needs no second tree.

Rules come from the frequent itemsets with minimum confidence 0.5 under the run's measure and are ranked by confidence, then lift, then support.

IV. Results

A. Dataset and preprocessing

Online Retail II [3] is a single workbook with two sheets, "Year 2009-2010" and "Year 2010-2011". Inspection showed that both sheets hold the invoices of 1 to 9 December 2010. The 22,523 rows (1,088 invoices) in this block are identical in both sheets, so we removed one copy. We then removed 6 accounting rows with invoice prefix A, 4,275 rows with no description, 5,771 rows with 25 non-product stock codes (postage, fees, manual adjustments, test items, gift vouchers), and 1,673 stock adjustment rows without a customer and with price 0 or less. Table I summarizes the result. Both runs use only baskets with a customer number, because a cancellation without one cannot be linked and both runs must mine the same baskets. This removes 2,922 purchase invoices (226,882 lines).

TABLE I
DATASET AFTER CLEANING

Quantity	Value
Raw rows (both sheets)	1,067,371
Purchase lines after cleaning	1,015,149
Baskets (invoices with customer)	36,597
Customers	5,853
Items (stock codes)	4,621
Cancellation lines	17,974
Date range	2009-12-01 to 2011-12-09
Basket size, mean / median / max	20.9 / 15 / 540
Train baskets (to 2010-11-30)	18,192
Hold-out baskets (from 2010-12-01)	18,405

Table II shows the linking outcome. Of 17,974 cancellation lines, 15,933 (88.6%) link fully and 117 link in part, and 451,890 of 469,882 cancelled units (96.2%) are allocated. 249 lines are split across more than one earlier purchase. The median lag from purchase to cancellation is 10.9 days. 35.5% of allocations happen within 7 days, 73.8% within 30 days, 90% within 90 days and 99.3% within 365 days. Under the policy "full", 5,872 of 766,150 basket items (0.77%) count as returned and 2,849 baskets (7.8%) contain a returned item. Under the policy "any", these are 15,654 items (2.04%) and 7,001 baskets (19.1%). The train line return rate is 0.83% and the hold-out rate is 0.70%.

TABLE II
RETURN LINKING OUTCOME

Outcome	Lines
Linked	15,933
Partially linked (quantity exceeds earlier purchases)	117
Unlinked: customer never bought the code	1,036
Unlinked: no purchase before the cancellation	498
Unlinked: no customer number	332
Unlinked: earlier purchases already returned	58

B. Correctness (E7)

All 29 tests pass. They include hand-built linking cases (several purchases, partial returns, a return larger than any single purchase line, a cancellation before any purchase, a missing customer), property tests with random baskets that check $\mathrm{rsup}(X) \leq \mathrm{sup}(X)$ and anti-monotonicity, and exact agreement with mlxtend [7] on toy data and on a seeded sample of 3,000 real train baskets. On the full train set, Apriori and FP-Growth return identical itemsets with identical counts at all nine minimum supports under both measures (18 of 18 settings). mlxtend's Apriori and FP-Growth find the same number of itemsets as our standard mode at every minimum support, for example 11,567 at 0.003.

C. Cost of the return-aware count (E1, E2)

We chose the minimum support grid 0.04, 0.03, 0.02, 0.015, 0.01, 0.0075, 0.005, 0.004 and 0.003 from item frequencies on the train baskets. Only 23 items reach support 0.05, so higher values give single items only. At 0.003 the baseline finds 11,567 itemsets of up to 7 items and Apriori stays within a 60 s budget per run. All runs used one 16-thread laptop with 7.6 GB of RAM. Runtime is the median of three runs measured with a performance counter. Peak memory is the tracemalloc peak in a separate untimed run.

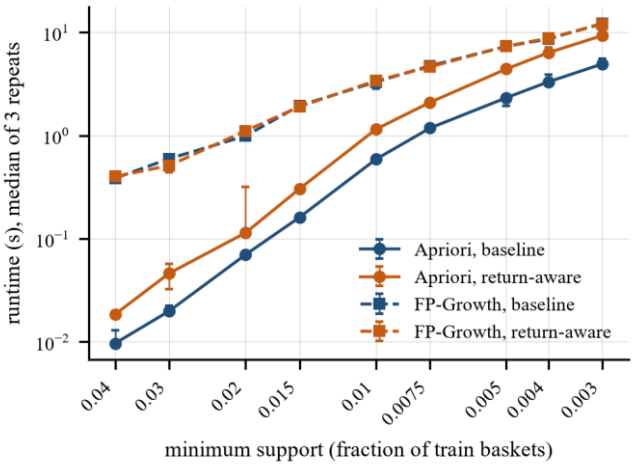


Fig. 2. Runtime against minimum support. Bars show the range of three runs.

TABLE III
RUNTIME (S) AND PEAK MEMORY (MIB)

Miner, minsup	Baseline	Return-aware	Ratio
Apriori, 0.01	0.59 s	1.15 s	1.95
Apriori, 0.005	2.32 s	4.41 s	1.90
Apriori, 0.003	4.93 s	9.29 s	1.88
FP-Growth, 0.01	3.30 s	3.40 s	1.03
FP-Growth, 0.005	7.30 s	7.32 s	1.00
FP-Growth, 0.003	12.19 s	11.99 s	0.98
Apriori memory, 0.003	22.5	41.3	1.84
FP-Growth memory, 0.003	100.4	113.0	1.13

Table III gives runtime and memory at three minimum supports. The return-aware Apriori takes 1.63 to 2.32 times as long as the baseline across the grid, because each candidate needs a second bitset intersection and a second bitset kept in memory. The return-aware FP-Growth takes 0.86 to 1.12 times as long. Its tree is built from kept baskets, which are slightly smaller, and the one extra count per output itemset is cheap. Our Apriori is faster than our FP-Growth at every minimum support. As a reference on the same machine, mlxtend needs 26.6 s (Apriori) and 7.0 s (FP-Growth) at 0.003. Our FP-Growth is a pure Python tree, which explains the gap to mlxtend.

D. Output change (E3)

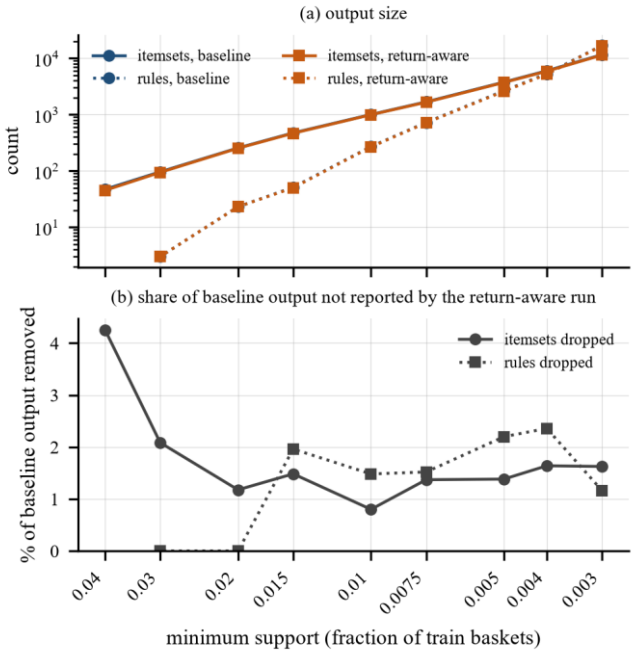


Fig. 3. (a) Itemsets and rules against minimum support. (b) Share of baseline itemsets and rules that the return-aware run does not report.

Fig. 3 shows that the two measures give almost the same output size. Since $\mathrm{rsup}(X) \le \mathrm{sup}(X)$, the return-aware itemsets are a subset of the baseline itemsets at every threshold, and between 0.8% and 4.3% of baseline itemsets disappear. Rules can also appear, because rconf can exceed conf when items of the antecedent were returned. Table IV gives the detail at three minimum supports.

TABLE IV
BASELINE AGAINST RETURN-AWARE OUTPUT

minsup	0.01	0.005	0.003
Itemsets, baseline	1,000	3,761	11,567
Itemsets dropped	8	52	188
Rules, baseline	270	2,638	16,850
Rules dropped	4	58	196
Rules added	0	8	66
Top-10 / top-50 overlap	0.90 / 0.96	1.00 / 1.00	1.00 / 0.92
Kendall tau (conf, rconf)	0.982	0.983	0.986

The Kendall tau-b between conf and rconf over the rules both runs report is between 0.979 and 0.986 for every minimum support of 0.02 or lower ($p < 0.001$ each). The top-ranked rules do not change because they have almost no returns. At minimum support 0.005 the top ten rules under both measures link pieces of one children's garden set (blue and pink trowels, forks and gloves), with confidence between 0.987 and 1.0 and lift near 79. Nine of them have rr = 0 and the tenth has rr = 0.007. The rules that drop out sit near a threshold. Of the 58, 47 lose support: their itemset falls below 0.005 on kept baskets, and 46 of these had baseline support below 0.0055. The other 11 lose confidence: their conf between 0.500 and 0.514 falls below 0.5. The 8 added rules have conf just below 0.5 and rconf of 0.5 or slightly above.

E. Return rates (E4)

We call an itemset return rate high when it is at least twice the train line return rate, so the threshold is 0.0166. Among the 3,761 baseline itemsets at minimum support 0.005, 1,722 (45.8%) have rr = 0, the median rr is 0.0038 and the 90th percentile is 0.0177. 426 itemsets (11.3%) and 174 of 2,638 baseline rules (6.6%) are high.

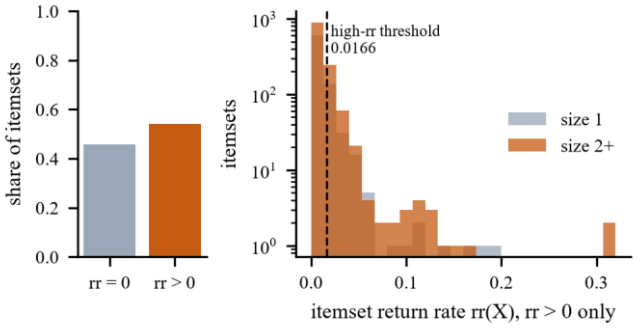


Fig. 4. Distribution of rr(X) over baseline itemsets at minimum support 0.005. Left: share with no returns. Right: itemsets with rr > 0, log scale.

Table V lists rules with the largest rr. The pair PINK CHERRY LIGHTS ⇒ WHITE CHERRY LIGHTS has rr = 0.32: in 32% of the baskets that held both, at least one of them was fully returned, and confidence falls from 0.658 to 0.554. The rule BAKING SET SPACEBOY DESIGN ⇒ BAKING SET 9 PIECE RETROSPOT has rr = 0.155 and falls from conf 0.508 to rconf 0.459, below the 0.5 cut, so the return-aware run does not report it.

TABLE V
RULES WITH HIGH ITEMSET RETURN RATE, MINSUP 0.005

Rule	conf	rconf	rr
Pink cherry lights ⇒ white cherry lights	0.658	0.554	0.320
Light pink cherry lights ⇒ white cherry lights	0.679	0.556	0.315
Baking set spaceboy ⇒ baking set retrospot	0.508	0.459	0.155
Regency cakestand + pink teacup ⇒ green teacup	0.926	0.922	0.053
Regency cakestand + green teacup ⇒ pink teacup	0.739	0.728	0.053

F. Hold-out stability (E5, E5b)

We took the top 10, 20 and 50 rules of each run on train and computed rconf on the kept baskets of the hold-out period. Because the top-50 lists are identical at minimum support 0.005, both runs give the same result: 100% of the top-k rules still reach rconf 0.5 on hold-out, and mean confidence changes by −0.028 (top-10), −0.058 (top-20) and −0.065 (top-50). The same holds on a censored hold-out of 11,829 baskets that ends 90 days before the end of the data, so late purchases that had little time to be returned do not drive this result.

A sharper test looks at the rules the runs disagree on (Table VI, Fig. 5a). Under the policy "full", 80.5% of the 2,580 rules found by both runs reach rconf 0.5 on hold-out. Of the 58 baseline-only rules, the rules that the return-aware run removed, 81.0% still do. None of the 8 return-aware-only rules does. Under the policy "any", the removed rules hold slightly less often than the shared ones (74.6% against 80.5%), and 2 of the 10 added rules hold. On this dataset, removing rules by retained support did not remove rules that fail later. The rules it added were borderline and failed more often.

TABLE VI
HOLD-OUT SHARE OF RULES WITH RCONF OF 0.5 OR MORE

Group (minsup 0.005)	Policy full	Policy any
Rules in both runs	0.805 (2,580)	0.805 (2,300)
Baseline-only rules	0.810 (58)	0.746 (338)
Return-aware-only rules	0.000 (8)	0.200 (10)

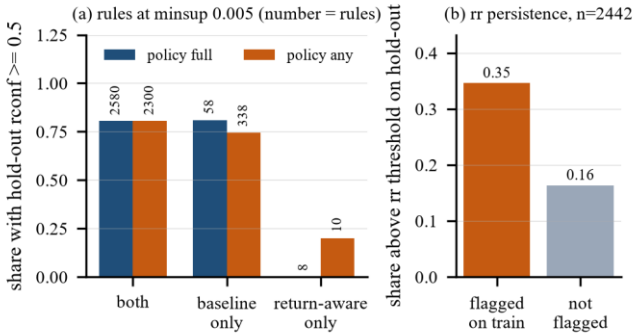


Fig. 5. (a) Share of rules that reach rconf 0.5 on hold-out, by group. (b) Share of itemsets above the rr threshold on hold-out, by flag on train.

E4b asks whether a high return rate on train predicts a high return rate later. Over 2,442 baseline itemsets of size 2 or more with at least 30 hold-out baskets, the Spearman correlation between train rr and hold-out rr is 0.10 ($p < 0.001$). The 269 itemsets flagged high on train have a median hold-out rr of 0.0120 against 0.0041 for the others (one-sided Mann-Whitney U, $p < 0.001$), and 34.6% of them stay above the threshold on hold-out against 16.3% of the others (Fig. 5b). The flag carries information, but a weak correlation means most flagged itemsets do not stay high.

G. Sensitivity (E6)

Table VII repeats the comparison at minimum support 0.005 with the other partial-return policy and with anonymous purchases included as baskets with k(t) = t ("anon."). With customers only the baseline reports 2,638 rules, and with anonymous purchases it reports 5,035.

TABLE VII
SENSITIVITY AT MINSUP 0.005

Variant	Line rr	Itemsets dropped	Rules dropped	Top-20	Tau
full (default)	0.83%	52	58	1.00	0.983
any	2.20%	270	338	0.75	0.952
full + anon.	0.67%	81	71	1.00	0.986
any + anon.	1.76%	449	603	0.70	0.950

The policy decides how much the output changes. With "any", 7.2% of baseline itemsets and 12.8% of baseline rules drop out, and a quarter of the top-20 rules change. Including anonymous purchases adds 1,551 train baskets, mostly large ones, and doubles the number of itemsets. The rule set then differs strongly from the default (Jaccard 0.41 between the return-aware rule sets), but this comes from the extra baskets, not from returns, since their kept baskets equal their full baskets.

V. Limitations and Challenges

Low return rate under the default policy. Only 0.77% of basket items count as returned under the policy "full". The approach changes little when returns are rare or when they fall mostly on items that rarely co-occur. Top rules in this dataset are bundles of matching variants, such as a garden set in two colors, and customers kept them. A retailer with a higher return rate, such as the fashion settings in [22], may see larger changes. We did not test this.

Linking is inferred. The data has no reference from a cancellation to its invoice. Our rule assigns a return to the most recent earlier purchase of the same code, which can be wrong when a customer bought the code several times. 1,924 cancellation lines (10.7%) stay unlinked, 1,036 of them because the customer never bought that code in the data. Some of these probably reverse purchases made before December 2009, which the data does not contain.

Partial returns. A cancellation often covers part of the quantity bought. 9,782 basket items received a partial allocation. The policy "full" ignores them and "any" counts them as returned, and E6 shows that this choice changes the output more than anything else we varied. A quantity-weighted measure could use the fractions, but a fractional count needs a new anti-monotonicity argument.

Right censoring. Purchases late in the data have had less time to be returned. 90% of returns happen within 90 days, so we repeated E5 on a hold-out that ends 90 days early. The result did not change, but the last weeks of the train period are affected in the same way and we did not correct for that.

Anonymous purchases. 2,922 invoices without a customer number cannot be linked to returns. Including them as baskets without returns understates returns and changes the rule set for reasons unrelated to returns.

Evaluation. Hold-out rconf measures whether a rule keeps its confidence, not whether a bundle built from it would sell or be returned. We have no data on business outcomes. Runtime numbers come from one laptop and a Python implementation, so only the ratios between measures on the same machine carry over.

Challenges. The workbook duplicated nine days of invoices across its two sheets, which we found only by checking invoice numbers across sheets. The first full run showed a peak of 1,043 MiB for return-aware Apriori at minimum support 0.005, because surviving itemsets held references to whole blocks of candidate bitsets. Copying only the surviving rows reduced it to 13 MiB, and we repeated the run. FP-Growth in pure Python needed care to avoid reading an input stream twice, a bug the tests caught.

VI. Conclusions and Future Work

Key findings. Retained support fits into Apriori and FP-Growth without changing either search procedure. Both algorithms agree exactly under both measures, and the baseline matches mlxtend. The extra count costs Apriori about twice the runtime and costs FP-Growth nothing measurable. On Online Retail II, under the default policy, retained support removes 0.8% to 4.3% of itemsets and up to 2.4% of rules, and leaves the top rules unchanged. The itemset return rate finds a minority of patterns, 6.6% of rules at minimum support 0.005, whose items come back at more than twice the usual rate, such as the cherry light pairs with rr = 0.32. That signal persists weakly into the following year. Removing rules by retained support did not make the remaining rules more stable on the hold-out period.

Implications. On this dataset, a retailer gains little from replacing support with retained support for ranking rules. The value lies in reporting rsup and rr next to sup and conf, so that an analyst can see which bundles customers keep and which they send back. The cost of computing both counts is small with FP-Growth.

Significance. The work turns the cancellation rows that basket analyses of this data usually delete into a measure, and shows that a return-aware count keeps the pruning property that utility measures lose.

Future work. With more time we would (1) test the measure on data with higher return rates, such as fashion, (2) develop a quantity-weighted retained support for partial returns and study its monotonicity, (3) model the uncertainty of the link between a return and a purchase, in the spirit of uncertain-data mining [17], [18], and (4) evaluate rules by later sales and returns of the bundles built from them, not only by confidence.

References

[1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules in large databases," in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, Santiago, Chile, 1994, pp. 487-499.

[2] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Dallas, TX, USA, 2000, pp. 1-12, doi: 10.1145/335191.335372.

[3] D. Chen, "Online Retail II," UCI Machine Learning Repository, 2012, doi: 10.24432/C5CG6D.

[4] shahashmi, "market-basket-analysis," GitHub repository. [Online]. Available: https://github.com/shahashmi/market-basket-analysis

[5] D. Karl, "Forecasting e-commerce consumer returns: A systematic literature review," *Manage. Rev. Quart.*, vol. 75, no. 3, pp. 1-56, 2025, doi: 10.1007/s11301-024-00436-x.

[6] Q. H. Duong, L. Zhou, M. Meng, T. V. Nguyen, P. Ieromonachou, and D. T. Nguyen, "Understanding product returns: A systematic literature review using machine learning and bibliometric analysis," *Int. J. Prod. Econ.*, vol. 243, Art. no. 108340, 2022, doi: 10.1016/j.ijpe.2021.108340.

[7] S. Raschka, "MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack," *J. Open Source Softw.*, vol. 3, no. 24, Art. no. 638, 2018, doi: 10.21105/joss.00638.

[8] W. Gan, J. C.-W. Lin, P. Fournier-Viger, H.-C. Chao, V. S. Tseng, and P. S. Yu, "A survey of utility-oriented pattern mining," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1306-1327, 2021, doi: 10.1109/TKDE.2019.2942594.

[9] P. Ambilkar, V. Dohale, A. Gunasekaran, and V. Bilolikar, "Product returns management: A comprehensive review and future research agenda," *Int. J. Prod. Res.*, vol. 60, no. 12, pp. 3920-3944, 2022, doi: 10.1080/00207543.2021.1933645.

[10] G. Shang, E. C. McKie, M. E. Ferguson, and M. R. Galbreth, "Using transactions data to improve consumer returns forecasting," *J. Oper. Manage.*, vol. 66, no. 3, pp. 326-348, 2020, doi: 10.1002/joom.1071.

[11] K. Meißner, A. B. Stevenson, and J. Rieck, "Identifying non-intuitive relationships within returns data of a furniture online-shop using temporal data mining," in *Recent Challenges in Intelligent Information and Database Systems (ACIIDS 2022)*, Commun. Comput. Inf. Sci., vol. 1716. Singapore: Springer, 2022, pp. 299-312, doi: 10.1007/978-981-19-8234-7_24.

[12] J. M. Luna, P. Fournier-Viger, and S. Ventura, "Frequent itemset mining: A 25 years review," *WIREs Data Mining Knowl. Discovery*, vol. 9, no. 6, Art. no. e1329, 2019, doi: 10.1002/widm.1329.

[13] L. Bustio-Martínez, R. Cumplido, M. Letras, R. Hernández-León, C. Feregrino-Uribe, and J. Hernández-Palancar, "FPGA/GPU-based acceleration for frequent itemsets mining: A comprehensive review," *ACM Comput. Surv.*, vol. 54, no. 9, 2022, doi: 10.1145/3472289.

[14] I. D. Hunyadi, N. Constantinescu, and O.-A. Ţicleanu, "Efficient discovery of association rules in e-commerce: Comparing candidate generation and pattern growth techniques," *Appl. Sci.*, vol. 15, no. 10, Art. no. 5498, 2025, doi: 10.3390/app15105498.

[15] W. Gan, S. Wan, J. Chen, C.-M. Chen, and L. Qiu, "TopHUI: Top-k high-utility itemset mining with negative utility," in *Proc. IEEE Int. Conf. Big Data*, 2020, pp. 5350-5359, doi: 10.1109/BigData50022.2020.9378288.

[16] J. He, X. Han, X. Wan, and J. Wang, "Efficient skyline frequent-utility itemset mining algorithm on massive data," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3009-3023, 2024, doi: 10.1109/TKDE.2024.3349454.

[17] Y. Tong, L. Chen, Y. Cheng, and P. S. Yu, "Mining frequent itemsets over uncertain databases," *Proc. VLDB Endow.*, vol. 5, no. 11, pp. 1650-1661, 2012, doi: 10.14778/2350229.2350277.

[18] Z. Li, F. Chen, J. Wu, Z. Liu, and W. Liu, "Efficient weighted probabilistic frequent itemset mining in uncertain databases," *Expert Syst.*, vol. 38, no. 5, Art. no. e12551, 2020, doi: 10.1111/exsy.12551.

[19] F. Zhao, D. Agrawal, A. El Abbadi, and A. Metwally, "SpaceSaving±: An optimal algorithm for frequency estimation and frequent items in the bounded-deletion model," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1215-1227, 2022, doi: 10.14778/3514061.3514068.

[20] T. Martin, G. Francoeur, and P. Valtchev, "CICLAD: A fast and memory-efficient closed itemset miner for streams," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 1810-1818, doi: 10.1145/3394486.3403232.

[21] J. Li, J. He, and Y. Zhu, "E-tail product return prediction via hypergraph-based local graph cut," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2018, pp. 519-527, doi: 10.1145/3219819.3219829.

[22] A. Nestler, N. Karessli, K. Hajjar, R. Weffer, and R. Shirvany, "SizeFlags: Reducing size and fit related returns in fashion e-commerce," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2021, pp. 3432-3440, doi: 10.1145/3447548.3467160.

[23] D. Chen, S. L. Sain, and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," *J. Database Marketing Customer Strategy Manage.*, vol. 19, no. 3, pp. 197-208, 2012, doi: 10.1057/dbm.2012.17.